

Generic external axis over Modbus TCP — protocol

One register map for any external axis a PLC fronts: a linear track carrying the robot (Pamet: Kinco PLC → Lichuan servo), a rotary positioner carrying the part, or any drive behind a PLC that can serve Modbus TCP. rw2 owns the axis through this map alone. The robot controller's own external-axis coupling is never used (owner ruling 2026-09-28/29). The map is the contract between the driver in this repository and the PLC program; both sides are written against this file.

Lineage: epic-065 external-axis-motion (Delta VFD-C2000 positioner, `delta-positioner` `DeltaRegisters.cs`, FR-11 dead-commander watchdog) and epic-065 feature-007 (Pamet track directly from the Kinco PLC).

Transport

Item	Value
Transport	Modbus TCP, port 502 (configurable)
Unit id	1 (configurable)
Register type	Command block: holding registers (FC03 read, FC06/FC16 write). Status block: input registers (FC04 read). No coils. Ownership is enforced by the function code, not by a promise: nobody can write where the PLC publishes. This is also how a CODESYS Modbus TCP slave maps its two register arrays, each from 0.
Block placement	Two blocks, each in its own address space: the command block at holding base <code>C</code> (default 0), the status block at input base <code>S</code> (default 0). Offsets inside a block are fixed; only the two bases are configurable, on both sides, for a PLC whose arrays cannot start at 0. <code>C+n</code> is holding register <code>C+n</code> and <code>S+n</code> is input register <code>S+n</code> ; the tables give the offset <code>n</code> .
Word order for 32-bit values	low word first (register <code>n</code> = bits 0–15, register <code>n+1</code> = bits 16–31), two's complement; each 16-bit register big-endian on the wire (Modbus standard)
Position unit	0.001 mm for a linear axis, 0.001° for a rotary axis (the driver's configured kind decides; the map is the same)
Velocity unit	0.001 unit/s; acceleration 0.001 unit/s²
Consistency	The PLC answers one FC04 read of the whole status block (15 registers) from one scan's image , and applies one FC16 write before or after a scan, never in the middle of one.
Map version	<code>MapVersion</code> (<code>S+14</code>) = 1 for this document. Amended in place on 2026-10-01 (status block to input registers at 0) because no PLC implemented it yet.

Command block — written by the driver, read by the PLC (holding registers `C+0 ... C+11`)

Addr	Name	Type	Meaning
0	<code>Command</code>	bitfield	bit 0 Enable (level: 1 = servo on). bit 1 Home , bit 2 MoveAbsolute , bit 3 MoveVelocity , bit 4 Stop , bit 5 Reset — edge-triggered: the PLC acts on 0→1 and the driver clears the bit after <code>CommandAck</code> echoes <code>CommandSeq</code> . Stop has priority over every other bit in the same word. Bits 6–15 are 0.
1	<code>CommandSeq</code>	uint16	Incremented by the driver with every command write — a write that sets an edge bit or changes Enable (never 0; 65535 wraps to 1). The write that only clears edge bits after an ack does not increment it.
2–3	<code>TargetPosition</code>	int32	Target for MoveAbsolute, in position units.
4–5	<code>Velocity</code>	int32	Speed for MoveAbsolute (magnitude) and MoveVelocity (signed: sign = direction).
6–7	<code>Acceleration</code>	int32	Ramp for the next move; 0 = PLC default.
8	<code>Heartbeat</code>	uint16	FR-11: the driver writes an incrementing value (1...65535, never 0) at ≥ 5 Hz for the whole connection lifetime. The driver's default is 10 Hz.
9	<code>LeaseOwner</code>	uint16	FR-11 advisory lease: 0 = unowned; else the station-unique owner id of the driver holding the axis.
10	<code>WatchdogFault</code>	uint16	0 healthy; 1 = the PLC tripped the watchdog. Cleared only by the driver writing 0 . Written by the PLC on trip.
11	<code>WatchdogTrips</code>	uint16	Trip counter since PLC power-up. PLC-owned, read-only for the driver.

Status block — written by the PLC, read by the driver (input registers S+0 ... S+14)

Addr	Name	Type	Meaning
0	State	uint16	0 Disabled · 1 Standstill · 2 Homing · 3 DiscreteMotion · 4 ContinuousMotion · 5 (reserved) · 6 Stopping · 7 ErrorStop — the numbers of the SDK <code>AxisState</code> enum. Any other value is a Protocol error (<code>ProtocolMismatch</code>).
1	Flags	bitfield	bit 0 Homed · bit 1 InPosition · bit 2 LimitMin · bit 3 LimitMax · bit 4 HomeSensor · bit 5 DriveReady · bit 6 Moving
2–3	ActualPosition	int32	Position units. Valid only while <code>Homed</code> is set (an absolute encoder keeps it across power cycles; the map does not care which).
4–5	ActualVelocity	int32	Signed, 0.001 unit/s.
6	FaultCode	uint16	0 none · 1 drive fault · 2 limit switch tripped · 3 following error · 4 watchdog · 5 homing failed · 6 communication to drive lost · 7 safety stop (E-stop chain, guard) · 100+ vendor-specific (documented per PLC).
7	CommandAck	uint16	Echo of the last accepted <code>CommandSeq</code> .
8–9	TravelMin	int32	Machine limit, published by the PLC. The driver refuses targets outside <code>TravelMin..TravelMax</code> ; it never clamps.
10–11	TravelMax	int32	
12–13	MaxVelocity	int32	Machine limit, published by the PLC. The driver refuses faster speeds.
14	MapVersion	uint16	1. The driver reads the status block before its first write and refuses to attach to any other value, writing nothing.

Limits come from the machine through this block. If a PLC cannot publish them (`TravelMin`, `TravelMax` and `MaxVelocity` all zero), the driver uses the values from its own configuration and logs that it did; it never invents a number. When they are published they must satisfy `TravelMin < TravelMax` and `MaxVelocity > 0`; any other combination (a partial publication included) makes the driver refuse to attach.

Command semantics

- **Handshake.** Before the command write the driver writes the parameters it needs (`TargetPosition`, `Velocity`, `Acceleration`, registers 2–7) in one FC16, then `Command` and `CommandSeq` (registers 0–1) in a second FC16 — so the PLC never sees an edge before its parameters. The PLC accepts a command word whose `CommandSeq` differs from `CommandAck` and copies `CommandSeq` into `CommandAck` **in the same scan in which it enters the resulting state** (Homing, DiscreteMotion, ContinuousMotion, Stopping, Disabled, Standstill). A status read that shows the new ack therefore also shows the command's state, never the state from before it.
- **Acknowledge:** a command is accepted when `CommandAck == CommandSeq`. The driver waits ≤ 500 ms for the ack; no ack $\rightarrow$ SDK `NotAcknowledged` for the command (the heartbeat keeps running); the driver clears the edge bit it set. At the deadline the driver reads the status block once more before declaring `NotAcknowledged`.
- **Sequence at attach:** the driver reads `CommandAck` and continues from `CommandAck + 1`, so a new driver never issues a sequence number the PLC already acknowledged.
- **Enable** (level): `Command.bit0 = 1` $\rightarrow$ PLC energises the drive; State leaves Disabled for Standstill. `bit0 = 0` $\rightarrow$ drive off, State = Disabled. After a watchdog trip or any entry into `ErrorStop` the PLC ignores bit 0 until a Reset, and after the Reset it energises only on a fresh `0  $\rightarrow$  1` of bit 0 — an axis never re-energises by itself. A move with Enable low is ignored and answered by `FaultCode = 0`, State = Disabled; the driver rejects it before writing (SDK `MotionError.Busy`). A `Velocity` above `MaxVelocity` or a `TargetPosition` outside `TravelMin..TravelMax` is acknowledged and ignored (State unchanged); a PLC never clamps.
- **Home** (edge): PLC runs its own homing sequence; State = Homing until done, then Standstill with `Homed`. The driver treats Home as callable from Disabled (it is the energise act: the driver sets Enable first). Failure $\rightarrow$ `ErrorStop`, `FaultCode = 5`.
- **MoveAbsolute** (edge): requires `Homed`; PLC moves to `TargetPosition` at `Velocity`; State = DiscreteMotion, then Standstill with `InPosition`. Target outside travel $\rightarrow$ the driver refuses (SDK `OutOfRange`) and writes nothing. There is deliberately no `MoveRelative` bit: the driver sends MoveAbsolute with `target = ActualPosition +  $\Delta$` , so the PLC never sees a relative command; the axis must be at Standstill and `Homed`, and unhomed jogging is `MoveVelocity`.
- **MoveVelocity** (edge): continuous motion at signed `Velocity` until Stop or a limit; State = ContinuousMotion. While `Homed`, reaching `TravelMin/TravelMax` is a controlled stop to Standstill (not a fault). A limit switch is always a fault: `ErrorStop`, `FaultCode = 2`.

- **Stop** (edge, priority): accepted in every state, Homing included; decelerate and hold; State = Stopping → Standstill. From Disabled or Standstill it is acknowledged and changes nothing. The driver sends Stop on the priority lane (see FR-11) and expects motion to cease within 200 ms of the write.
- **Reset** (edge): clears `FaultCode` and leaves `ErrorStop` → Disabled (Enable must be re-asserted, see Enable). Does not clear `WatchdogFault` (register 10 is cleared explicitly) and does not touch `Homed`. From any state other than `ErrorStop` it is acknowledged and changes nothing.
- **Limit switch recovery**: after a Reset with a limit switch still active, the PLC accepts motion only in the direction away from that switch.

FR-11 — dead-commander watchdog and advisory lease (verbatim from epic-065)

Motion must cease when the commander **dies**, not only when it calls Stop. Modbus TCP has no liveness semantics; a dead commander is indistinguishable from an idle one. Therefore:

- The driver runs a **connection-lifetime heartbeat** per axis, started at connect, stopped deliberately at disconnect, independent of motion, writing `Heartbeat` at ≥ 5 Hz. Heartbeat traffic shares the channel with move traffic; heartbeat deferral is **bounded at 200 ms** so a long move cannot starve the beat.
- The PLC **arms** the watchdog on the **first change** of `Heartbeat` it observes while `WatchdogFault` = 0 and `LeaseOwner` $\neq$ 0 (so an idle PLC is not faulted one second after boot), then **trips** when `Heartbeat` has not changed for **1 s**: Enable dropped, velocity zeroed, State = `ErrorStop`, `FaultCode` = 4, `WatchdogFault` = 1, `WatchdogTrips` += 1. A trip does **not** touch `Homed` (recovery = Reset + re-command, no re-home) and re-asserts the limit functions. After a trip the network disarms until `WatchdogFault` has been written 0 and a new beat arrives.
- **Clean release disarms**. When `LeaseOwner` is written 0 the watchdog disarms without tripping. A clean disconnect is: Stop → Enable 0 → stop beating → `LeaseOwner` = 0. `WatchdogTrips` therefore counts dead commanders only. A killed process cannot write 0, so the kill case is untouched.
- **Advisory lease**: before attaching, the driver reads `LeaseOwner`. 0 → take it (write own id). Own id → reattach. Foreign id → watch `Heartbeat` at the heartbeat interval; unchanged for one expiry window (1 s) → the lease has expired, take it; still changing → refuse (SDK `LeaseHeld`), keep watching until the configured lease timeout. Watching continuously rather than in discrete 1 s samples lets a successor attach ≤ 1.1 s after the incumbent's last beat. "Advisory" is stated plainly: Modbus has no compare-and-swap, so two drivers starting in the same window can both pass; the watchdog bounds the consequence. On a clean disconnect the driver writes `LeaseOwner` = 0 if it still holds it.
- **At attach** the driver writes `WatchdogFault` = 0 after winning the lease: a latched trip then belongs to a dead predecessor, and leaving it latched would leave the network disarmed for the new commander. `FaultCode` = 4 and `ErrorStop` stay until a Reset, so the trip remains visible and distinguishable from a drive fault.
- Owner id is a station-unique non-zero 16-bit value from host configuration.
- Each heartbeat tick is: write `Heartbeat`, read holding C+9...C+11, read the status block (one FC04 of S+0...S+14). That tick is the status cadence of the axis, moving or idle.
- Stop and cancellation take a **priority lane** on the channel that preempts queued move traffic (≤ 200 ms to a halted axis).

What the PLC program must do (checklist for the ladder / structured-text author)

1. Serve holding registers C+0...C+11 and input registers S+0...S+14 on the configured unit id and port; refuse nothing, ignore writes to holding C+11. The status block is input registers and cannot be written.
2. Publish `MapVersion` = 1, `TravelMin/Max`, `MaxVelocity` from the axis parameters at start-up.
3. Mirror the drive: `State`, `Flags`, `ActualPosition`, `ActualVelocity`, `FaultCode` every PLC cycle; serve a status-block read from one scan's image.
4. Execute the command semantics above; echo `CommandSeq` into `CommandAck` in the scan that enters the command's state.
5. Implement the watchdog exactly as FR-11 states (arm on first change with a lease held, 1 s stall, trip actions, disarm until cleared, disarm on `LeaseOwner` = 0).
6. Pass the conformance checks below, CHK-01...CHK-16 (motion checks at commissioning, with an operator present).
7. Keep the project in a git repository next to this file's copy (epic-065 risk R-1: an unversioned PLC program is a drive that answers every register and moves nothing after replacement).

Scope of map version 1

- One axis per block pair. A PLC serving two axes places a second block pair at other bases and is configured as two devices.

- A rotary axis has limited travel: `ActualPosition` is absolute and never wraps. Endless rotation is not in version 1.

Errors and debugging

Owner rulings 2026-09-29: an error must say what was seen, and never claim a cause it did not observe. There are four error classes. The driver, both checkers, the reports and the logs use the same four names, and nothing translates an error from one class into another.

Class	Meaning	SDK MotionError (2.30.0)	Who acts
Transport	The link failed: the TCP connect failed or was refused, the socket closed, a request got no answer within 500 ms, or the PLC returned Modbus exception 04 (slave device failure), 06 (busy), 0A/0B (gateway) or any code other than 01–03.	<code>CommunicationLost</code>	Network, IP, port, unit id.
Protocol	The PLC answered, but not per this document.	<code>ProtocolMismatch</code> : <code>MapVersion</code> $\neq$ 1 · limits partial or not sane · <code>State</code> is 5 or above 7 · <code>State</code> = 7 with <code>FaultCode</code> = 0 · the PLC changed a driver-owned register · the PLC returned Modbus exception 01 (illegal function), 02 (illegal data address) or 03 (illegal data value), with no retry. <code>NotAcknowledged</code> : a command was written, and <code>CommandAck</code> did not echo <code>CommandSeq</code> within 500 ms.	PLC programmer.
Machine	The PLC reports a fault, or the machine did not do what the PLC accepted.	<code>FaultCode</code> 1 → <code>DriveFault</code> · 2 → <code>LimitTripped</code> · 3 → <code>MotionFailed</code> · 4 → <code>WatchdogTripped</code> · 5 → <code>HomeLatchFailed</code> · 6 → <code>DriveFault</code> (drive link) · 7 → <code>SafetyStop</code> · $\geq$ 100 → <code>DriveFault</code> (vendor code in the message). An accepted command that misses the driver's budget: no Standstill after Enable → <code>DriveFault</code> ; homing not finished → <code>HomeLatchFailed</code> ; stopped outside the in-position window, or not arrived, or still moving after Stop → <code>MotionFailed</code> .	Maintenance or operator.
Commander	The driver refused before writing anything.	<code>Busy</code> , <code>NotHomed</code> , <code>OutOfRange</code> , <code>UnreachableSpeed</code> , <code>UnsupportedSense</code> , <code>LeaseHeld</code> , <code>UnknownAxis</code> , <code>WrongAxisKind</code> (binding refusals)	The caller, or the other commander.

Every `MotionError` member of SDK 2.30.0 appears in exactly one row. A member added later is unmapped until this table names its class.

Rules

1. **Say what you saw.** Every error message and every FAIL line has this shape: `<axis or CHK-nn>: <Class>/<MotionError>: <what happened>`. Read `<Register> (<address>) = <value>[, expected <value>]`. `<address>` is the offset and the absolute register with its type: `C+n` = holding a, `S+n` = input a (a = base + n). A command error adds `CommandSeq <n>` written, `CommandAck <m>` read, `State <s>` read. A transport error adds the endpoint, the operation, the register range and the exception message. A bare "communication error" is a defect.
2. **One cause, one class.** A Protocol or Machine error is never reported as Transport. A Transport failure is never reported as a Machine fault. After a link loss the driver reports `CommunicationLost`. When the link returns, a watchdog trip the PLC reports is a separate `WatchdogTripped`, and the log carries both.
3. **No silent recovery.** One retry of a Transport failure, after a reconnect, is the only retry; a Protocol refusal is never retried. A refusal identical to the previous tick's logs at Warning once, then at Debug until the cause changes or the link recovers; the latched error itself is logged once at Error. The retry logs at Warning with the exception, and so does every failed heartbeat beat. A command is never re-sent, and an ack is waited for once.
4. **A register dump comes first.** Both checkers take `--dump`. It reads holding `C+0...C+11` and input `S+0...S+14` once, or at 5 Hz with `--watch` until Ctrl-C. It prints one row per register: address, name, raw hex, and the decoded value in engineering units, with `State` and `FaultCode` names and `Flags` and `Command` bits by name. `--dump` writes nothing and takes no lease. Its exit code is 0 when both blocks were read, and 1 on a Transport or Protocol error.
5. **Every FAIL carries evidence.** A FAIL in a report attaches the last read of both blocks, decoded as by `--dump`.

Example messages:

```

carriage: Protocol/ProtocolMismatch: attach refused. Read MapVersion (S+14 = input 14) = 2, expected 1.
carriage: Protocol/NotAcknowledged: Home not accepted. CommandSeq 42 written, CommandAck 41 read after 500 ms, State 0 read.
carriage: Machine/WatchdogTripped: Read FaultCode (S+6 = input 6) = 4, WatchdogFault (C+10 = holding 10) = 1, WatchdogTrips (C
+11 = holding 11) = 3.
carriage: Transport/CommunicationLost: FC04 read S+0...S+14 on 192.168.58.20:502 unit 1 failed twice (reconnected once): Connec
tion refused.
carriage: Protocol/ProtocolMismatch: FC04 read S+0...S+14 = input 0...14 refused: Modbus exception 02 (illegal data address) - t
he PLC does not serve the status block as input registers.
CHK-06: Protocol/NotAcknowledged: Enable 1 not accepted. CommandSeq 7 written, CommandAck 6 read after 500 ms, State 0 read.

```

Conformance checks

This checklist is executable. Two tools run it verbatim against a PLC, with no rw2 involved:

- the C# test app, `dotnet ModelingEvolution.GenericAxis.TestApp.dll --check <host>[:port] ...`;
- the Python tool, `python -m generic_axis_check <host>[:port] ...` (in `python/generic_axis_check/`).

The simulator must pass the full list, motion included, in CI. The table below is the single source of truth. A check id missing from either tool, or present in a tool but not here, is a defect, and each tool has a test that compares its ids with this table.

Command line (identical in both tools)

Argument	Default	Meaning
<code><host>[:port]</code>	port 502	The PLC.
<code>--unit N</code>	1	Unit id.
<code>--command-base N</code> , <code>-</code> <code>--status-base N</code>	0, 0	Block bases: holding C, input S.
<code>--owner-id N</code>	65535	The checker's lease id. CHK-11 uses 65534 as the foreign id. Ids 65534–65535 are reserved for conformance tools; stations never use them.
<code>--allow-motion</code>	off	Runs CHK-12...CHK-16. Without it they are SKIPPED. Only with an operator at the machine and the travel clear.
<code>--tolerance X</code>	0.1	Position check threshold in axis units (CHK-13). This is a checker threshold, not a machine number.
<code>--dump [--watch]</code>	off	Prints the decoded register dump (§ Errors and debugging, rule 4) and runs no checks.
<code>--command <verb></code> <code>[args] [--for S]</code>	off	Runs one verb instead of the checks (One-verb mode, below): <code>enable</code> , <code>disable</code> , <code>home</code> , <code>stop</code> , <code>reset</code> , <code>move <target> [--speed <pct>]</code> , <code>jog <signed velocity></code> . <code>home</code> , <code>move</code> and <code>jog</code> need <code>--allow-motion</code> . <code>--for S</code> ends a jog after S seconds.
<code>--report PATH</code>	none	<code>*.md</code> : writes the Markdown report there and the JSON report next to it as <code>*.json</code> . <code>*.json</code> : writes the JSON report only. Any other extension is a usage error (exit 2). The Markdown report always goes to stdout.

Exit codes: 0 = no FAIL (SKIPPED allowed) · 1 = at least one FAIL · 2 = usage error · 3 = refused to start (another commander is beating — rw2, a station, or a second tool; see Pre-flight; stop it first) · 4 = interrupted by the operator (Ctrl-C / SIGINT).

One-verb mode (`--command`), for commissioning: send one verb and watch it, with no checklist. It writes no report; `--command` with `--dump` or `--report` is a usage error. Both tools do exactly this:

- Run the pre-flight of § Rules for every run (exit 3, `RESULT: REFUSED`, if another commander beats).
- Read the status block. `MapVersion` $\neq$ 1 or limits not sane → Protocol error, exit 1, nothing written.
- Run the guards on that read (Commander class, exit 2; the message names the register and its value): `move` needs `Homed` and a target inside `TravelMin..TravelMax`; `--speed` defaults to 10 % of `MaxVelocity` and must be $0 < \text{pct} \leq 100$. **Speed rounding** (both modes, both tools): a percentage becomes `raw = round-half-away-from-zero(pct × MaxVelocity raw ÷ 100)`; a raw 0 is refused, never floored. The refusal body is exactly `<pct> % of MaxVelocity rounds to raw Velocity 0 (round-half-away-from-zero(<pct> × <raw> ÷ 100) = 0)`; nothing to move with. Read `MaxVelocity` (`S+12 = input 12`) = `<raw>`. (the address renders as rule 1; input 12 at the default base). `<pct>` is the percentage as given (typed with `--speed`, e.g. 0.00005, or the check's 10 / 1), printed in full and never rounded for display. This guard prints `Commander/UnreachableSpeed: refused before writing anything: <body>`. `jog` needs $0 < |v| \leq \text{MaxVelocity}$, `v` in axis units/s. Unpublished limits (all zero) refuse `move` and `jog`. A refused guard writes nothing at all: no lease, no beat. Only then take the lease under the tool's id and beat every 100 ms. After a dead holder's trip, also write `WatchdogFault = 0`, as the driver does at attach, leaving `ErrorStop` and `FaultCode = 4` for reset.

4. Send the verb through the handshake of § Command semantics: parameters in one FC16, then `Command + CommandSeq` in a second FC16; `ack` ≤ 500 ms; the edge cleared after the `ack`. `enable`, and `home`, `move` and `jog` from Disabled, set Enable first, as the driver's Home does, with an Enable 0 first when bit 0 already reads 1 (a fresh edge, as the driver energises); "found energised" is decided from State, never from the command bit; cleanup writes Enable 0 only if the axis was found not energised and this run wrote Enable 1. `reset` writes `WatchdogFault = 0` if set and Enable 0 before the Reset edge.
5. Print every status read at 20 ms (`State`, `Flags`, `ActualPosition`, `ActualVelocity`, `FaultCode`, `CommandAck`) until the verb completes: `enable` → Standstill (5 s) · `disable` → Disabled (5 s) · `home` → Standstill + Homed (120 s) · `stop` → Standstill or Disabled (5 s) · `reset` → not ErrorStop (5 s) · `move` → Standstill + InPosition ($2 \times |\Delta| \div v + 5$ s) · `jog` → ContinuousMotion observed (500 ms after the `ack`), then printing continues until `--for` elapses or Ctrl-C, and the tool sends Stop. Each budget is measured from the completion of the verb's write (`jog`'s 500 ms from the `ack`), as in § Rules for every run > Timing.
6. Clean up as § Rules for every run: Stop if moving, clear edges, Enable 0 by the rule of step 4, release the lease. A run leaves the axis no more energised than it found it: `enable` proves the handshake and ends Disabled.

Exit codes: 0 = the verb completed (a jog ended by `--for` or Ctrl-C after ContinuousMotion was observed included) · 1 = the PLC failed it (Transport, Protocol or Machine, in the shape of § Errors and debugging rule 1) · 2 = usage error or a guard refused · 3 = refused by pre-flight · 4 = interrupted by the operator before the verb completed. The last line is `RESULT: PASS` (exit 0), `RESULT: FAIL` (exit 1), `RESULT: GUARD` (exit 2, a guard refused), `RESULT: REFUSED` (exit 3) or `RESULT: INTERRUPTED` (exit 4); a usage error prints no `RESULT` line.

Rules for every run

- **Pre-flight.** Before its own first beat, the tool reads `LeaseOwner` and watches `Heartbeat` (C+8) for 1 s. Any change of `Heartbeat` in that window, whatever `LeaseOwner` holds (0, a station id, or the tool's own id), means another commander is live. The tool then writes nothing, reports every check SKIPPED, names the observed beat values and `LeaseOwner` in the message, sets `summary.result` to REFUSED (Markdown last line `RESULT: REFUSED`), and exits 3. A refused run never reports PASS. This also catches a second conformance tool using the same owner id. A held lease is watched longer, because a commander silent for 1 s may still be alive (the watchdog trips only 1.0–1.5 s after its last beat). When `LeaseOwner` ≠ 0, the tool also reads `WatchdogFault` (C+10) and watches for at least 1 s and up to 1.6 s (1.5 s plus one 100 ms read):
 - `Heartbeat` changes at any time: refused, as above. A beat after a trip is a live commander.
 - `WatchdogFault` reads 1 and `Heartbeat` did not change for the full 1 s watch: the lease holder is dead. The tool proceeds, and the line after the Markdown heading says "Pre-flight: `LeaseOwner` (C+9 = holding 9) = n held with no beat and `WatchdogFault` (C+10 = holding 10) = 1: the previous commander is dead; its trip is left for its operator." The tool never clears that trip: a restore that finds it FAILs `Machine/WatchdogTripped`, and cleanup leaves it and the lease as they were.
 - Neither within 1.6 s: refused (exit 3, REFUSED), naming `LeaseOwner` and `WatchdogFault`: "a live commander, or a PLC without a working watchdog; release `LeaseOwner` by hand only if no commander runs".

With `LeaseOwner = 0` the watch stays 1 s. A pre-flight read that fails (after the one retry) never proved the axis free: `CHK-01 FAILs Transport/CommunicationLost` with that read's message, every later check is SKIPPED ("needs `CHK-01`, which FAILED"), and nothing is written.

- **Isolation.** Each tool run uses its own working directory for logs and reports. A run against a simulator uses a simulator on its own port. Two concurrent runs never share a PLC, a simulator or a report path.
- **Order.** Checks run in id order. A check whose prerequisite FAILED or was SKIPPED is SKIPPED with the message `needs CHK-n, which FAILED or needs CHK-n, which SKIPPED`, naming the first such prerequisite in Needs order.
- **Timing.** Timing checks poll the status block every **20 ms**. Every duration is measured from the completion of the triggering write to the first read that shows the effect, and is reported in ms. A window's bounds are judged at that read cadence: an effect happened between the last read without it and the first read with it, so it is early only if the first read with it is before the window, and late only if the last read without it is already after it.
- **Units.** All positions and velocities are raw register values ÷ 1000, in the PLC's axis unit (mm or °).
- **Cleanup, always, even after a FAIL or Ctrl-C:** Stop edge if State is 2, 3 or 4 · clear edge bits · Enable 0 · `WatchdogFault = 0` if the checker caused a trip · `LeaseOwner = 0` if it holds the checker's id. The beat continues through cleanup and stops just before `LeaseOwner = 0`. A cancellation never interrupts a frame in flight: the request completes or times out first, so the connection stays usable for cleanup. Each cleanup write is logged in the report.
- **Lease and beat between checks.** From the end of pre-flight onwards the checker holds the lease under its own id and beats: it writes `LeaseOwner = its id` and starts its beat as soon as pre-flight passes and `MapVersion` (S+14) reads 1, so a second tool is refused from `CHK-01` on. The exception is where a check says it stops. After a dead holder's trip (Pre-flight) it takes no lease and does not beat. If a beating checker reads `LeaseOwner` ≠ its own id, the running check FAILs `Protocol/ProtocolMismatch` "Read `LeaseOwner` (C+9 = holding 9) = n, expected 65535" (the register does not hold what was written), every later check is SKIPPED with that reason, and the checker writes nothing more to that axis except to stop its own beat.

- **Each check restores.** Every check ends with the axis in State 0 or 1, no latched fault, the lease held and the beat running: Reset, `WatchdogFault = 0` and re-take as needed. If it cannot restore, it FAILs with the reason, and every later check is SKIPPED. A precondition FAIL is a failure to restore.
- **Beat.** Whenever a check says "beat", the checker writes `Heartbeat` every 100 ms (1...65535, never 0) from its own loop, not through a driver.

The checks

Id	Title	Protocol section	Needs	Procedure	PASS when
CHK-01	Transport and unit	Transport	—	TCP connect (a budget of ≤ 3 s including the tool's own retry; a budget, not a timing threshold), then FC04 of S+0...S+14 on the unit.	Connected, and the read answers with no Modbus exception.
CHK-02	Map version	Status block	01	Read S+14 (FC04).	<code>MapVersion == 1</code> .
CHK-03	Machine limits published	Status block, "Limits come from the machine"	02	Read S+8...S+13 (FC04).	Not all zero, <code>TravelMin < TravelMax</code> , <code>MaxVelocity > 0</code> . The values are reported.
CHK-04	Status mirror cadence	Status block; FR-11 tick	02	30 status-block reads (FC04), one every 100 ms.	All 30 answer, the slowest round trip is ≤ 100 ms, and <code>State</code> $\in \{0,1,2,3,4,6,7\}$ in every read.
CHK-05	32-bit word order and driver ownership of parameters	Transport (word order); Command block	02	Write C+2...C+3 = <code>[0x0002, 0x0001]</code> (65 538), read back. Write -2 as <code>[0xFFFE, 0xFFFF]</code> , read back. Wait 1 s and read again. No edge bit is set.	Both values read back exactly (FC03), and are unchanged after 1 s (the PLC does not write driver-owned holding registers). The PLC's <i>interpretation</i> of the order is proven by CHK-03 (sane limits) and CHK-13 (it arrives where it was sent).
CHK-06	Enable handshake (level)	Command semantics: Handshake, Enable	02	Precondition State 0 or 1. Take the lease (checker id), beat. Write <code>[Enable, seq+1]</code> , then after the ack <code>[0, seq+2]</code> . If Command bit 0 already reads 1 while State is 0, write <code>[0, seq+1]</code> first and wait for its ack, so the Enable is a fresh 0 $\rightarrow$ 1 edge (the later numbers shift by one).	Each ack arrives in ≤ 500 ms. State is 1 within 5 s after Enable 1 and 0 within 5 s after Enable 0. Ack ms and state ms are reported. Energises the drive and commands no motion.
CHK-07	Reset handshake (edge)	Command semantics: Reset, Acknowledge	06	From State 0 write <code>[Reset, seq+1]</code> . After the ack, clear the edge <code>[0, seq+1]</code> .	Ack in ≤ 500 ms, <code>State</code> stays 0 and <code>FaultCode</code> stays 0 (Reset outside <code>ErrorStop</code> is a no-op).
CHK-08	Watchdog trips on a stalled beat	FR-11	06	Hold the lease, <code>WatchdogFault = 0</code> , beat for 2 s, then stop beating. Keep polling.	<code>WatchdogFault == 1</code> , <code>WatchdogTrips +1</code> , <code>State == 7</code> , <code>FaultCode == 4</code> , all within 1.0–1.5 s of the last beat. The trip time is reported.
CHK-09	Watchdog disarms after a trip and re-arms on clear	FR-11	08	Setup: first trip the watchdog as in CHK-08 (lease held, <code>WatchdogFault = 0</code> , beat 2 s, stop, wait ≤ 1.5 s; no trip $\rightarrow$ FAIL "setup: no trip"). Then, without clearing, beat 1 s: no second trip is counted. Then Reset edge, <code>WatchdogFault = 0</code> , beat 2 s (must not trip), stop beating.	No trip while latched. No trip while beating. A second trip within 1.0–1.5 s with <code>WatchdogTrips +1</code> . Recovery afterwards is Reset plus <code>WatchdogFault = 0</code> .
CHK-10	Clean release disarms	FR-11 "Clean release disarms"	08	Beat 2 s, write <code>LeaseOwner = 0</code> , stop beating, wait 2 s.	No trip: <code>WatchdogFault == 0</code> and the trip count is unchanged.

Id	Title	Protocol section	Needs	Procedure	PASS when
CHK-11	Advisory lease	FR-11 "Advisory lease"	02	(a) LeaseOwner = 0 → the checker's lease client takes it and reads back its id, then releases. (b) Write LeaseOwner = 65534 and beat as that incumbent. The checker's lease client, with a 3 s timeout, must refuse. (c) Stop the incumbent's beat while the client watches.	(a) Read-back equals the checker id. (b) Refused with LeaseHeld naming 65534 after 3 s, and LeaseOwner is still 65534. (c) Taken within 2 s of the incumbent's last beat, with the time reported. Any trip caused by (c) is cleaned up. Before restoring after (c), the checker waits until WatchdogFault reads 1 or 1.6 s have passed since the incumbent's last beat, then clears it.
CHK-12	Home	Command semantics: Home	06, --allow-motion	Enable, then Home edge.	Ack in ≤ 500 ms. Then State == 1 with Homed within 120 s, and FaultCode == 0. Duration reported.
CHK-13	MoveAbsolute to TravelMin + 10	Command semantics: MoveAbsolute	03, 12	Target TravelMin + 10, velocity 10 % of MaxVelocity (Speed rounding, § One-verb mode), acceleration 0.	Ack with State == 3 in ≤ 500 ms. Then State == 1 with InPosition, arriving within $2 \times target - start \div velocity + 5$ s (start = ActualPosition before the move), and $abs(ActualPosition - target) \leq --tolerance$. The error and duration are reported.
CHK-14	Stop mid-move	Command semantics: Stop; FR-11 priority	13	MoveAbsolute toward TravelMin + (TravelMax - TravelMin)/2 at 10 % (Speed rounding). Once $abs(ActualVelocity) \geq 90$ % of the commanded speed (or after 2 s), write Stop.	Ack in ≤ 500 ms. ActualVelocity == 0 and State == 1 within 200 ms of the Stop write. The time is reported.
CHK-15	MoveVelocity	Command semantics: MoveVelocity	13	MoveVelocity at +1 % of MaxVelocity (Speed rounding; away from TravelMin) for 1 s, then Stop.	Ack with State == 4, ActualVelocity > 0 during the run, then State == 1 after Stop within 200 ms.
CHK-16	Kill test	FR-11	08, 15	MoveVelocity at +1 % (Speed rounding); ActualVelocity > 0 within 2 s of the ack, then stop beating. The connection stays open, and polling continues.	Trip (FaultCode 4, State 7) within 1.0–1.5 s of the last beat. ActualVelocity == 0 within 200 ms of the trip. Homed is still set. Both times are reported.

Error class of a FAIL

Every FAIL carries one class (a checker error carries none, § Report schema), decided by what the checker saw, in this order (§ Errors and debugging):

Seen	Class / name
A request got no answer, the connect failed, the socket closed, or the PLC returned a Modbus exception other than 01–03 (after the one retry)	Transport / CommunicationLost
A command was written and CommandAck did not echo CommandSeq within 500 ms	Protocol / NotAcknowledged
State is 5 or above 7, or State = 7 with FaultCode = 0	Protocol / ProtocolMismatch
The PLC reports State = 7 with a FaultCode ≠ 0 where the check did not expect a fault	Machine / the FaultCode map (for example 1 → DriveFault)
The PLC answered, but against this document: Modbus exception 01, 02 or 03 in any check (no retry); CHK-02, 03, 04 (an invalid State, or the slowest round trip over 100 ms), 05, 07 (Reset changed the axis); any watchdog behaviour in CHK-08, 09, 10 and CHK-16's trip (no trip, early or late trip, a trip while latched or after release, Homed cleared by a trip); CHK-11 when a register does not hold what was written, or Heartbeat keeps changing after the incumbent stopped writing it; an ack read without the command's state (CHK-12...15, "ack in the scan that enters the state")	Protocol / ProtocolMismatch
An accepted command whose effect never came: no Standstill after Enable 1 or no Disabled after Enable 0 (CHK-06) → DriveFault; not homed within 120 s (CHK-12) → HomeLatchFailed; not arrived, outside --tolerance, left ContinuousMotion, no velocity, or a halt over 200 ms (CHK-13...16) → MotionFailed	Machine

There is no Commander class in a checker FAIL: the checker writes raw registers and refuses nothing, except a speed that rounds to raw 0, which SKIPS that check with the refusal body alone (Speed rounding, no class prefix), writing nothing; `errorClass` stays `null`, as on every SKIP.

- **The one retry.** A checker performs the one reconnect-and-retry the driver performs, logs it at Warning, and counts it in that check's `retries` (§ Observed values). A second failure is a Transport FAIL. A Protocol refusal is not retried.
- `lastRead` is a fresh read of both blocks, taken when the failure is detected and before any restore write. If that read fails, it holds the last values read, with `null` for a register never read.
- **Interruption** is not a FAIL. The running check and every later one are `SKIPPED`, with the message "interrupted by the operator during CHK-*nn*". Cleanup runs, `summary.result` is `INTERRUPTED`, and the exit code is 4.

Observed values

`observed` is exactly the keys listed here for the check, in this order: nothing more, nothing less. Every value is an integer or `null`. `null` means the value was never observed, for example because the check failed before reaching it. A `SKIPPED` check has `observed: {}`. Every non-skipped check ends with `retries`, the number of reconnect-and-retries performed during it (normally 0).

Units: **ms**, a duration measured as in § Rules for every run. **raw**, an int32 register value (0.001 axis unit, or 0.001 unit/s for velocities). **reg**, a uint16 register value as read. **count**, a number of events.

Id	Keys (unit)
CHK-01	<code>connectMs</code> (ms), <code>readMs</code> (ms), <code>retries</code> (count)
CHK-02	<code>mapVersion</code> (reg), <code>retries</code>
CHK-03	<code>travelMin</code> (raw), <code>travelMax</code> (raw), <code>maxVelocity</code> (raw), <code>retries</code>
CHK-04	<code>reads</code> (count answered), <code>slowestMs</code> (ms), <code>invalidStates</code> (count), <code>retries</code>
CHK-05	<code>firstReadBack</code> (raw, after writing 65 538), <code>secondReadBack</code> (raw, after writing -2), <code>secondReadBackAfter1s</code> (raw), <code>retries</code>
CHK-06	<code>enableAckMs</code> (ms), <code>enableStateMs</code> (ms to State 1), <code>disableAckMs</code> (ms), <code>disableStateMs</code> (ms to State 0), <code>retries</code>
CHK-07	<code>ackMs</code> (ms), <code>state</code> (reg, after), <code>faultCode</code> (reg, after), <code>retries</code>
CHK-08	<code>tripAfterMs</code> (ms from the last beat), <code>watchdogTrips</code> (reg, after the trip), <code>faultCode</code> (reg), <code>state</code> (reg), <code>retries</code>
CHK-09	<code>setupTripAfterMs</code> (ms), <code>tripsWhileLatched</code> (count), <code>tripsWhileBeating</code> (count), <code>secondTripAfterMs</code> (ms), <code>watchdogTrips</code> (reg, at the end), <code>retries</code>
CHK-10	<code>tripsAfterRelease</code> (count), <code>watchdogFault</code> (reg, 2 s after release), <code>retries</code>
CHK-11	<code>ownIdReadBack</code> (reg), <code>refusedAfterMs</code> (ms until LeaseHeld), <code>leaseOwnerAfterRefusal</code> (reg), <code>takenAfterMs</code> (ms from the incumbent's last beat), <code>retries</code>
CHK-12	<code>ackMs</code> (ms), <code>homedAfterMs</code> (ms), <code>faultCode</code> (reg, at the end), <code>retries</code>
CHK-13	<code>target</code> (raw), <code>ackMs</code> (ms), <code>arrivedAfterMs</code> (ms), <code>position</code> (raw ActualPosition at rest), <code>positionError</code> (raw, absolute), <code>retries</code>
CHK-14	<code>commandedVelocity</code> (raw), <code>velocityAtStop</code> (raw ActualVelocity at the Stop write), <code>ackMs</code> (ms), <code>haltMs</code> (ms), <code>retries</code>
CHK-15	<code>commandedVelocity</code> (raw), <code>ackMs</code> (ms), <code>maxVelocitySeen</code> (raw), <code>stopAckMs</code> (ms), <code>haltMs</code> (ms), <code>retries</code>
CHK-16	<code>commandedVelocity</code> (raw), <code>tripAfterMs</code> (ms from the last beat), <code>haltAfterTripMs</code> (ms), <code>homedAfterTrip</code> (0 or 1), <code>retries</code>

Anything else a tool wants to say goes into `message`. The id-parity test on each side also compares these key lists with the tool's output.

Report schema

JSON (schema: "generic-axis-conformance/1"). Both tools emit exactly these fields:

```
{
  "schema": "generic-axis-conformance/1",
  "mapVersion": 1,
  "tool": { "name": "generic-axis-check", "language": "python", "version": "1.0.0" },
  "target": { "host": "192.168.58.20", "port": 502, "unit": 1, "commandBase": 0, "statusBase": 0 },
  "allowMotion": false,
  "startedAt": "2026-09-29T10:15:02Z",
  "finishedAt": "2026-09-29T10:15:31Z",
  "preflight": null,
  "summary": { "result": "PASS", "pass": 11, "fail": 0, "skipped": 5 },
  "checks": [
    { "id": "CHK-08", "title": "Watchdog trips on a stalled beat", "section": "FR-11",
      "result": "PASS", "durationMs": 3140,
      "message": "trip after 1.12 s", "errorClass": null,
      "observed": { "tripAfterMs": 1120, "watchdogTrips": 3, "faultCode": 4, "state": 7, "retries": 0 } },
    { "id": "CHK-06", "title": "Enable handshake (level)", "section": "Command semantics: Handshake, Enable",
      "result": "FAIL", "durationMs": 612, "errorClass": "Protocol",
      "message": "Protocol/NotAcknowledged: Enable 1 not accepted. CommandSeq 7 written, CommandAck 6 read after 500 ms, State 0 read.",
      "observed": { "enableAckMs": null, "enableStateMs": null, "disableAckMs": null, "disableStateMs": null, "retries": 0 },
      "lastRead": { "command": [1, 7, 0, 0, 0, 0, 0, 0, 12, 65535, 0, 2],
        "status": [0, 32, 0, 0, 0, 0, 0, 6, 0, 0, 38528, 152, 41248, 7, 1] } },
    { "id": "CHK-12", "title": "Home", "section": "Command semantics: Home",
      "result": "SKIPPED", "durationMs": 0, "message": "needs --allow-motion", "errorClass": null, "observed": {} }
  ],
  "cleanup": [ "C+0 = 0x0000 (Enable 0)", "C+9 = 0 (release lease)" ]
}
```

- `preflight` is null unless pre-flight had something to say; the same text is the line after the Markdown heading.
- `result` is PASS, FAIL or SKIPPED. `summary.result` is REFUSED if pre-flight refused to start (exit 3), otherwise INTERRUPTED if the operator interrupted the run, otherwise FAIL if any check failed, otherwise PASS.
- `errorClass` is Transport, Protocol or Machine on a FAIL, and null otherwise. `errorClass` is null on a FAIL only when the checker itself failed (message checker error: ...); such a run is not a verdict on the PLC and must be repeated after the tool is fixed. A FAIL also carries `lastRead`: the raw values of holding C+0...C+11 and input S+0...S+14, read as § Error class of a FAIL says.
- `observed` follows § Observed values exactly. `language` is python or csharp.

The Markdown report has four parts, in order:

1. A heading with the tool, the target and the UTC time.
2. A table with the columns Id | Title | Result | Observed | Protocol section.
3. **Failures**: for each FAIL, its message, then the `--dump` rendering of its `lastRead`.
4. The cleanup list.

The last line is RESULT: PASS, RESULT: FAIL, RESULT: INTERRUPTED or RESULT: REFUSED.

Reference: the Delta positioner ladder this generalises

```
M0 -> M1025 (RUN) + M1040 (servo on)
M4 -> edges: MOV 0/1 D1060 (speed/position mode); FREQ D110 D111 D112
M5 -> M1026 (direction: OFF = forward, ON = reverse)
M6 + edge X7 -> DSUB D1051 D120 D122; DMOV D1051 D120 (home latch)
D130 heartbeat · D131 lease owner · D132 latched watchdog fault · D133 trip counter
```

(delta-positioner/src/RocketWelder.SDK.Devices.Motion.Delta/DeltaRegisters.cs, epic-065 current-state.md.)